DOKUMEN EVALUASI PRAKTIKAL (ADVANCED ASSESSMENT) PROGRAM: JUNIOR CYBERSECURITY ETHICAL HACKER

PENYELENGGARA: PT VINIX SEVEN AURUM

Topik: Denial of Service (App-Layer), Evasion Detection, dan Mitigasi WAF

Target Sistem: Portal Web PT Teknologi Aman Sejahtera (Sistem Simulasi)

Tautan Lab:

Kelompok 8 :

- **Afnan Yazid Pradana**
- **Kelvin Febrianto**
- **Kurniawan Saputra**
- **Rishfan Mailizar**
- **Robin Felix Hama**

Cybersecurity Ethical Hacker

A. DESKRIPSI TUGAS Pada minggu ini, insiden siber berskala menengah sedang berlangsung. Portal utama perusahaan mengalami kelumpuhan intermiten (*intermittent downtime*), dan alarm dari sistem *Intrusion Detection System (IDS)* lama tidak mendeteksi anomali *payload*. Sebagai tim Analis Keamanan, kelompok Anda ditugaskan untuk mengunduh dan menganalisis dataset log mentah untuk membuktikan terjadinya *Application Layer DoS*, membongkar teknik pengelabuan (*evasion*) yang dilakukan penyerang, serta mengonfigurasi arsitektur pertahanan *Web Application Firewall (WAF)* untuk menangkal serangan lanjutan.

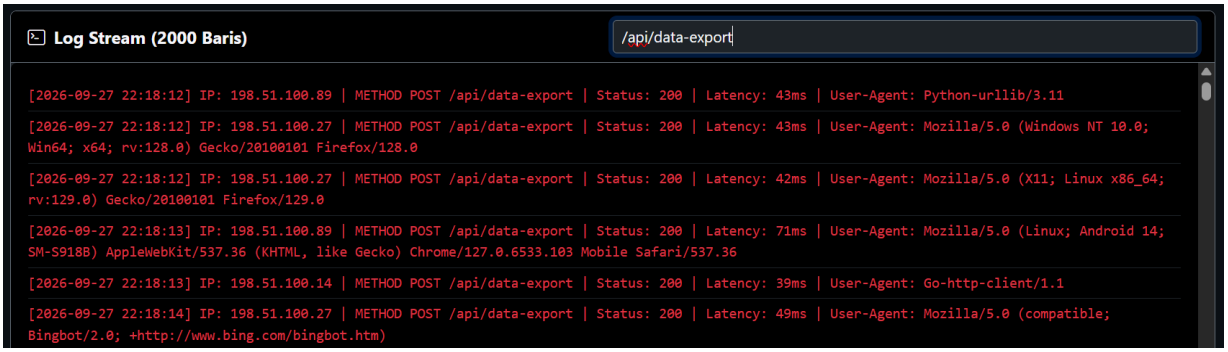
B. PETUNJUK PENGERJAAN

1. Evaluasi ini dikerjakan secara **berkelompok** untuk menguji ketajaman forensik log dan logika pertahanan infrastruktur tim Anda.
2. Anda diperbolehkan menggunakan *tools* pengolah teks (seperti `grep`, `awk`, Excel, atau script Python) untuk membedah *dataset* log.
3. Seluruh *screenshot* wajib menyertakan identitas peramban/terminal untuk membuktikan orisinalitas pengerjaan kelompok.
4. Laporan akhir dikumpulkan dalam format PDF dengan format penamaan berkas: `[Nama Kelompok]_[Nama Divisi]_Tugas Modul 5.pdf`

C. INSTRUKSI DAN RINCIAN TUGAS

BAGIAN 1: FORENSIK LALU LINTAS DATA (APP-LAYER DoS ANALYSIS) - **Bobot: 30%** Buka tautan lab dan unduh berkas dataset simulasi: `https://<URL_REPLIT_ANDA>/static/traffic_dataset.log` (atau salin langsung dari terminal server Anda).

1. **Identifikasi Anomali Trafik:** Lakukan analisis kuantitatif pada log tersebut. Identifikasi 3 (tiga) Alamat IP yang paling dominan mengirimkan *request* ke *endpoint* `/api/data-export`.



Jawaban:

Berdasarkan analisis kuantitatif terhadap request menuju endpoint /api/data-export, terdapat tiga alamat IP yang paling dominan, yaitu:

1. 198.51.100.27
2. 198.51.100.14
3. 198.51.100.89

Ketiga IP tersebut menghasilkan request HTTP POST menuju endpoint /api/data-export dengan pola trafik yang berulang. Endpoint tersebut juga merupakan endpoint yang relatif berat karena proses ekspor data umumnya membutuhkan pemrosesan database dan penggunaan resource server.

2. **Flash Crowd vs DoS:** Penyerang mengubah-ubah *User-Agent* pada setiap *request* (User-Agent Spoofing). Jelaskan berdasarkan bukti log, mengapa lonjakan trafik dari IP penyerang tersebut diklasifikasikan sebagai *Application-Layer DoS (HTTP Flood)*, dan bukan lonjakan pengguna sah (*Flash Crowd / Legitimate Traffic Burst*)?
   Jawaban:
   Lonjakan trafik dari IP 198.51.100.27, 198.51.100.14, dan 198.51.100.89 lebih sesuai dikategorikan sebagai Application-Layer DoS (HTTP Flood) daripada flash crowd berdasarkan pola request pada log.
   a. **Request berulang menuju endpoint yang sama**
      Seluruh contoh log menunjukkan request:

METHOD POST /api/data-export

Endpoint /api/data-export terus dipanggil secara berulang dalam rentang waktu yang sangat pendek. Pola seperti ini berbeda dari trafik pengguna normal yang biasanya mengakses berbagai halaman atau endpoint sesuai aktivitas pengguna.

b. User-Agent berubah-ubah secara tidak wajar

Contohnya, IP 198.51.100.27 menggunakan beberapa User-Agent berbeda hanya dalam beberapa detik:

- Firefox pada Windows
- Firefox pada Linux
- Bingbot
- Chrome pada Windows
- Chrome pada Android

Sementara itu, IP 198.51.100.89 juga berganti antara:

Python-urllib/3.11

Chrome pada Android

Perubahan User-Agent yang sangat cepat dapat menjadi indikator User-Agent spoofing/rotation, yaitu upaya membuat request otomatis terlihat seperti berasal dari berbagai jenis client.

c. Terdapat User-Agent yang menunjukkan HTTP client, bukan browser biasa

Contohnya:

Python-urllib/3.11

Go-http-client/1.1

Kedua User-Agent tersebut merupakan HTTP client yang dapat digunakan untuk menghasilkan request secara programatis. Kehadirannya dalam pola request berulang menuju endpoint yang sama memperkuat indikasi otomatisasi.

d. Request terjadi dengan interval sangat pendek

Contoh:

- 22:18:12 198.51.100.89
- 22:18:12 198.51.100.27
- 22:18:12 198.51.100.27
- 22:18:13 198.51.100.89
- 22:18:13 198.51.100.14
- 22:18:14 198.51.100.27
- 22:18:14 198.51.100.27
- 22:18:15 198.51.100.27

Dalam waktu sekitar 3 detik, sudah terdapat banyak request ke endpoint yang sama. Pola temporal yang padat dan repetitif seperti ini merupakan karakteristik request flooding.

e. Status HTTP 200 tidak berarti trafik tersebut legitimate

Semua contoh request mendapatkan:

Status: 200

Namun, HTTP 200 hanya menunjukkan bahwa server berhasil memproses request tersebut. Status tersebut tidak membuktikan bahwa request berasal dari pengguna sah. Justru pada application-layer DoS, penyerang dapat sengaja mengirim request yang valid secara sintaksis agar server tetap melakukan

proses aplikasi yang mahal.

f. Perbedaan dengan Flash Crowd

Pada flash crowd, peningkatan trafik biasanya berkaitan dengan aktivitas pengguna yang sah, misalnya banyak pengguna mengakses layanan karena suatu peristiwa populer. Biasanya terdapat variasi endpoint, sesi, pola navigasi, dan karakteristik client.

Pada dataset ini, terdapat kombinasi:

- request berulang ke endpoint spesifik;
- interval request sangat rapat;
- penggunaan HTTP client otomatis;
- pergantian User-Agent secara cepat;
- fokus terhadap endpoint /api/data-export.

Oleh karena itu, berdasarkan bukti log yang tersedia, pola tersebut lebih konsisten dengan Application-Layer DoS/HTTP Flood daripada legitimate traffic burst.

3. **Analisis Exhaustion:** Perhatikan metrik `Latency: <ms>` pada log serangan tersebut. Jelaskan secara teknis mengapa serangan pada *endpoint* spesifik (seperti ekspor data, pencarian, atau *login*) jauh lebih mematikan bagi *Resource* CPU/RAM Server dibandingkan serangan *Volumetric DDoS (Network Layer)* biasa.

Jawaban:

Serangan pada endpoint tertentu seperti data export, search, atau login dapat lebih membebani CPU dan RAM dibandingkan sekadar serangan volumetrik pada network layer karena setiap HTTP request yang berhasil mencapai application layer dapat memicu proses internal server.

Misalnya pada endpoint:

POST /api/data-export

satu request dapat menyebabkan aplikasi:

1. menerima dan memvalidasi request;
2. menjalankan middleware/autentikasi;
3. membuat query ke database;
4. mengambil sejumlah besar record;
5. melakukan filtering atau transformasi data;
6. membuat file CSV/Excel/JSON;
7. melakukan serialisasi data;
8. menyimpan hasil sementara di memory;
9. mengirimkan hasil kembali kepada client.

Jika satu request membutuhkan resource yang relatif besar, maka ratusan atau ribuanj request otomatis dapat menyebabkan resource exhaustion meskipun bandwidth jaringan tidak terlalu besar.

Pada log yang diberikan terdapat latency:

43 ms
43 ms
42 ms
71 ms
39 ms
49 ms
44 ms
72 ms

Nilai tersebut menunjukkan waktu yang dibutuhkan server untuk menyelesaikan request pada saat sampel tersebut dicatat. Latency yang lebih tinggi dibandingkan beberapa request lain dapat menjadi indikator bahwa sebagian request membutuhkan waktu pemrosesan lebih besar. Namun, latency saja belum cukup untuk membuktikan CPU/RAM exhaustion. Untuk membuktikan resource exhaustion secara kuat diperlukan metrik tambahan seperti:

1. CPU utilization;
2. RAM utilization;
3. database query time;
4. jumlah koneksi aktif;
5. request per second (RPS);
6. queue/thread utilization;
7. response time percentile, misalnya P95/P99;
8. jumlah error/timeout.

Mengapa berbeda dengan Volumetric DDoS?

Application-Layer DoS memiliki karakteristik yang berbeda dengan Volumetric DDoS karena target utama serangannya berada pada lapisan aplikasi, bukan hanya pada kapasitas jaringan. Pada Application-Layer DoS, penyerang biasanya mengirimkan request HTTP ke aplikasi atau API, misalnya melalui HTTP Flood pada endpoint seperti /api/data-export, /login, atau /search. Request tersebut dapat terlihat seperti request HTTP yang valid sehingga server tetap memprosesnya melalui berbagai komponen aplikasi.

Berbeda dengan itu, Volumetric DDoS lebih berfokus pada membanjiri jaringan dengan jumlah paket atau trafik yang sangat besar, misalnya melalui UDP flood atau TCP packet flood. Tujuan utamanya adalah menghabiskan kapasitas bandwidth atau membebani perangkat jaringan seperti router dan firewall sehingga trafik pengguna yang sah tidak dapat mencapai server. Serangan jenis ini tidak selalu menyebabkan proses aplikasi seperti query database atau pembuatan file dijalankan.

Pada Application-Layer DoS, resource yang dapat terkuras meliputi CPU, RAM, database, dan application worker. Setiap request yang berhasil mencapai aplikasi dapat menjalankan proses seperti autentikasi, validasi input, query database, pemrosesan data, hingga pembuatan response. Dampaknya terhadap database dan CPU aplikasi dapat menjadi signifikan, meskipun jumlah trafik yang dikirimkan oleh penyerang tidak sebesar trafik pada serangan volumetrik. Sebaliknya, pada Volumetric DDoS, dampak utama lebih banyak terjadi pada bandwidth dan infrastruktur jaringan, sedangkan database dan logika aplikasi belum tentu terlibat secara langsung.

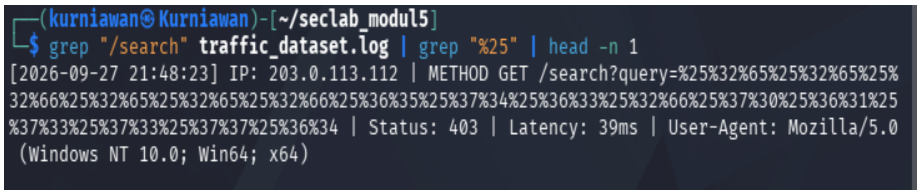
Hal yang membuat Application-Layer DoS sangat efektif adalah adanya

amplification pada level aplikasi. Penyerang dapat mengirimkan request HTTP yang ukurannya relatif kecil, tetapi request tersebut dapat menyebabkan server melakukan pekerjaan yang jauh lebih besar. Sebagai contoh, satu request ke /api/data-export dapat memicu query database, mengambil banyak record, melakukan pemrosesan atau transformasi data, menggunakan memory untuk menyiapkan hasil, dan kemudian menghasilkan response. Jika request tersebut dikirimkan berulang kali dalam jumlah besar, resource server seperti CPU, RAM, database connection, dan application worker dapat habis meskipun bandwidth yang digunakan penyerang relatif kecil.

Dengan demikian, perbedaan utamanya terletak pada resource yang menjadi sasaran. Volumetric DDoS berusaha menghabiskan kapasitas jaringan dengan volume trafik yang sangat besar, sedangkan Application-Layer DoS mengeksploitasi biaya pemrosesan aplikasi dengan mengirimkan request yang memaksa server melakukan pekerjaan internal yang relatif mahal.

BAGIAN 2: DEKONSTRUKSI TEKNIK EVASION - Bobot: 30% Pada log yang sama, terdapat anomali pada *endpoint* `/search` di mana *Intrusion Detection System (IDS)* perusahaan gagal memblokir serangan Injeksi (SQLi/LFI).

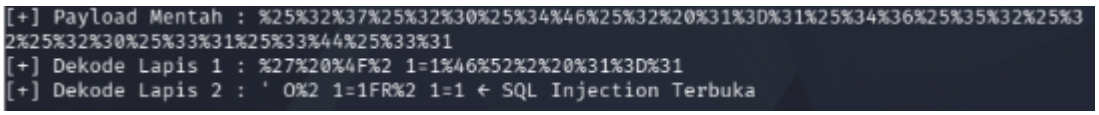
1. Temukan baris log yang mengandung kueri parameter dengan karakter `%` (Persen) yang panjang dan berulang. Salin baris log tersebut ke dalam laporan Anda.



Hasil pencarian log terhadap endpoint /search. Terlihat parameter query mengandung rangkaian karakter %25, %32, %33, dan karakter encoded lainnya. Pola tersebut menunjukkan bahwa payload tidak ditulis langsung dalam bentuk plaintext, melainkan telah melalui proses URL Encoding. Penggunaan encoding seperti ini merupakan teknik obfuscation/evasion karena karakter yang sebenarnya memiliki arti khusus bagi SQL atau aplikasi disamarkan menjadi bentuk encoded.

2. **Proses Dekode (De-obfuscation):** Lakukan proses *URL Decoding* pada *payload* tersebut. (Gunakan CyberChef atau *tools* sejenis).

a. Berapa lapis penyandian (*encoding*) yang digunakan oleh penyerang?



Pada screenshot berikutnya terlihat hasil analisis yang menunjukkan payload memiliki beberapa lapisan encoding. Terminal menampilkan informasi seperti Payload Mentah, Decode Lapisan 1, dan Decode Lapisan 2. Hal ini menunjukkan bahwa payload perlu didekode secara bertahap, bukan hanya sekali.

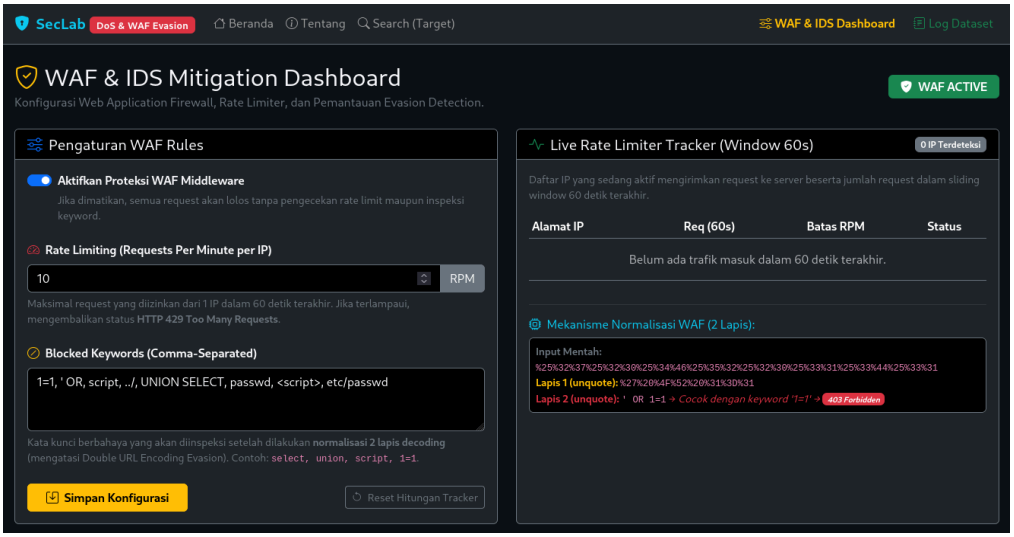
b.Apa hasil *payload* teks terang (*plaintext*) setelah berhasil didekode sepenuhnya

```
┌──(kurniawan㉿Kurniawan)-[~/seclab_modul5]
└─$ python3 -c 'import urllib.parse as u; p="%25%32%37%25%32%30%25%34%46%25%32%20%31%3D%3
1%25%34%36%25%35%32%25%32%25%32%30%25%33%31%25%33%44%25%33%31"; print("[+] Payload Mentah
 :", p); print("[+] Dekode Lapis 1 :", u.unquote(p)); print("[+] Dekode Lapis 2 :", u.unq
uote(u.unquote(p)), "← SQL Injection Terbuka")'
[+] Payload Mentah : %25%32%37%25%32%30%25%34%46%25%32%20%31%3D%31%25%34%36%25%35%32%25%3
2%25%32%30%25%33%31%25%33%44%25%33%31
[+] Dekode Lapis 1 : %27%20%4F%2 1=1%46%52%2%20%31%3D%31
[+] Dekode Lapis 2 : ' O%2 1=1FR%2 1=1 ← SQL Injection Terbuka
```

Screenshot ini memperlihatkan proses decoding menggunakan script Python. Program melakukan URL decoding secara berulang menggunakan fungsi seperti unquote() sampai payload tidak lagi berubah. Hasil decoding kemudian ditampilkan untuk mengetahui plaintext sebenarnya dari payload.

Hasil pada terminal menunjukkan bahwa setelah proses decoding, payload berubah dari rangkaian %xx menjadi teks yang dapat dibaca. Pada bagian ini, hasil plaintext digunakan sebagai bukti bahwa payload yang awalnya terlihat seperti data acak sebenarnya menyimpan pola injeksi. Ini merupakan inti dari teknik de-obfuscation

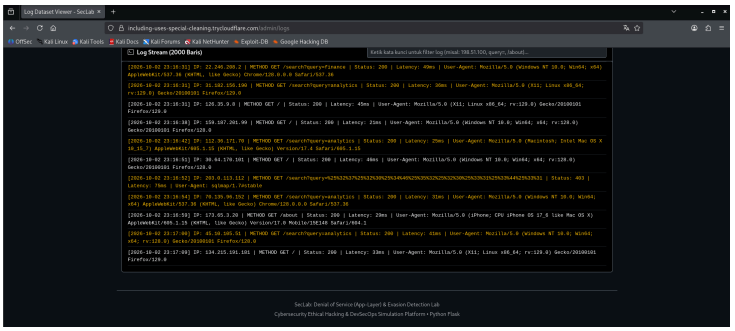
3. **Analisis Kegagalan IDS:** Mengapa teknik *encoding* (seperti URL/Hex Encoding) dan *fragmentation* sering kali berhasil mengecoh mesin IDS/IPS generasi lama yang hanya menggunakan pencocokan pola (*String/Signature Matching*)?



Pada dashboard terlihat daftar Blocked Keywords yang digunakan untuk mendeteksi pola payload berbahaya. Keyword plaintext dari hasil decoding Bagian 2 dimasukkan ke dalam daftar tersebut sehingga WAF dapat mengenali payload setelah dilakukan normalisasi/decoding.

```
┌──(kurniawan㉿Kurniawan)-[~/seclab_modul5]
└─$ for i in $(seq 1 20); do
echo -n "Request #$i: "
curl -s -o /dev/null -w "HTTP Status: %{http_code} | Latensi: %{time_total}s\n" \
-X POST https://including-uses-special-cleaning.trycloudflare.com//api/data-export
done
Request #1: HTTP Status: 200 | Latensi: 1.253771s
Request #2: HTTP Status: 200 | Latensi: 1.138065s
Request #3: HTTP Status: 200 | Latensi: 1.157117s
Request #4: HTTP Status: 200 | Latensi: 1.186828s
Request #5: HTTP Status: 200 | Latensi: 1.222960s
Request #6: HTTP Status: 200 | Latensi: 1.298794s
Request #7: HTTP Status: 200 | Latensi: 1.211919s
Request #8: HTTP Status: 200 | Latensi: 1.130452s
Request #9: HTTP Status: 200 | Latensi: 1.170856s
Request #10: HTTP Status: 200 | Latensi: 2.537274s
Request #11: HTTP Status: 429 | Latensi: 0.187040s
Request #12: HTTP Status: 429 | Latensi: 0.364946s
Request #13: HTTP Status: 429 | Latensi: 0.157612s
Request #14: HTTP Status: 429 | Latensi: 0.210363s
Request #15: HTTP Status: 429 | Latensi: 0.309085s
Request #16: HTTP Status: 429 | Latensi: 0.311706s
Request #17: HTTP Status: 429 | Latensi: 0.184292s
```

Screenshot terminal menunjukkan simulasi request berulang terhadap server. Beberapa request awal mendapatkan HTTP 200, kemudian setelah melewati batas tertentu respons berubah menjadi HTTP 429 (Too Many Requests). Ini menunjukkan bahwa mekanisme rate limiting berhasil membatasi jumlah request yang dapat diterima dalam periode tertentu.

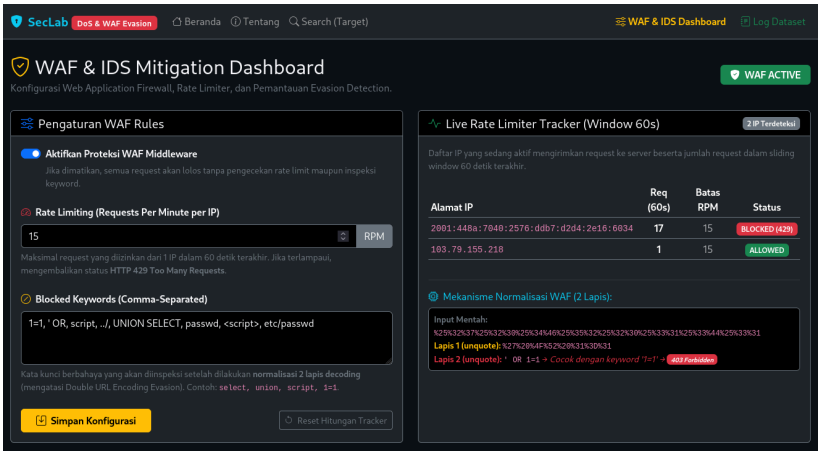


menampilkan antarmuka *Log Dataset Viewer* dari platform lab keamanan siber SecLab. Log server tersebut merekam aktivitas permintaan HTTP, di mana sistem berhasil mendeteksi dan memblokir (status **403 Forbidden**) upaya serangan *SQL Injection* otomatis menggunakan *tool* SQLMap pada alamat IP

BAGIAN 3: KONFIGURASI ARSITEKTUR MITIGASI (WAF & RATE LIMIT) - Bobot: 40% Masuk ke dasbor pertahanan perusahaan: `https://<URL_REPLIT_ANDA>/admin/waf-config`.

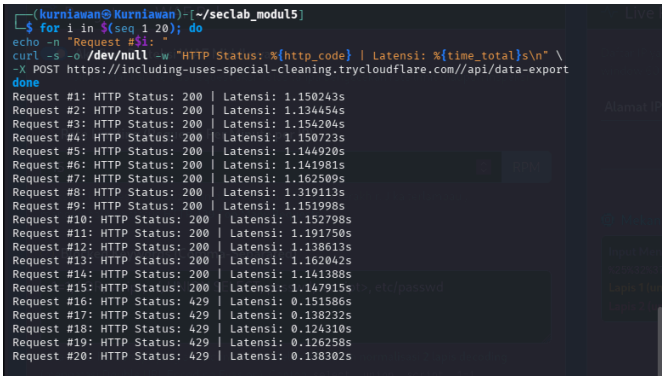
1. **Implementasi Rate Limiting:**

a. Konfigurasikan batas ambang wajar (*Rate Limit*) sebesar **15 Request per Menit** pada dasbor.

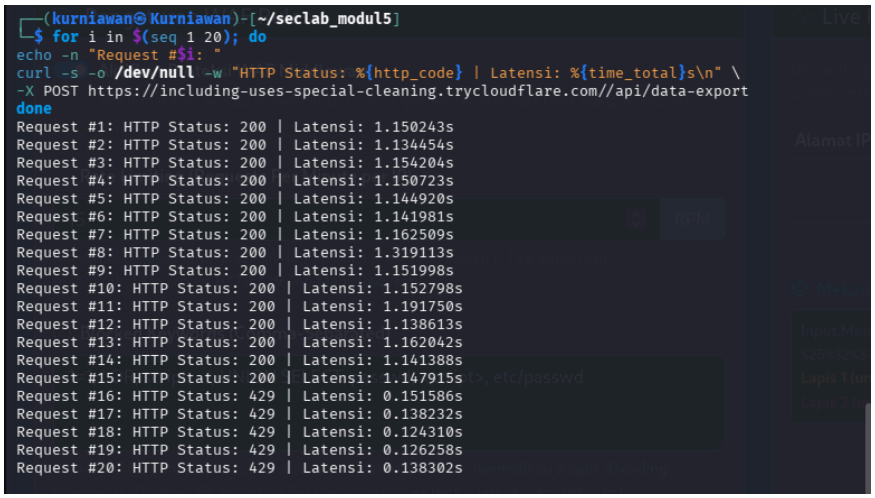


Screenshot terminal menunjukkan simulasi request berulang terhadap server. Beberapa request awal mendapatkan HTTP 200, kemudian setelah melewati batas tertentu respons berubah menjadi HTTP 429 (Too Many Requests). Ini menunjukkan bahwa mekanisme rate limiting berhasil membatasi jumlah request yang dapat diterima dalam periode tertentu.

b. Jalankan script `attack_sim.py` (yang diberikan instruktur) di terminal lokal Anda.



c. Lampirkan *screenshot* yang membuktikan bahwa *request* ke-16 ditolak oleh sistem dengan kode status *HTTP 429 (Too Many Requests)*. Jelaskan kelebihan dan kelemahan *Rate Limiting* berbasis IP.



Skrip ini mensimulasikan uji coba Rate Limiting (pembatasan frekuensi akses). Pada permintaan ke-16 dan seterusnya, server menolak akses karena mendeteksi jumlah pengiriman permintaan yang terlalu cepat dalam satu jendela waktu, yang menandakan bahwa mekanisme pelindung (Rate Limiter) pada lab berjalan dengan baik.

2. **Implementasi Normalisasi & Pemblokiran Evasion:**

a. Pada dasbor WAF, tambahkan kata kunci *plaintext* dari *payload* yang Anda temukan di Bagian 2 (misalnya `1=1` atau `script`) ke dalam daftar "Blocked Keywords".

```
┌──(kurniawan㉿Kurniawan)-[~/seclab_modul5]
└─$ curl -s "https://including-uses-special-cleaning.trycloudflare.com/search?query=%25%32
%37%25%32%30%25%34%46%25%35%32%25%32%30%25%33%31%25%33%44%25%33%31"
{"client_ip":"2402:8780:106b:8ce7:67ed:3537:bd97:3e3","error":"Forbidden - WAF Rule Violat
ion","matched_keyword":"1=1","message":"Potential Evasion/Attack Detected! Keyword terlara
ng '1=1' ditemukan setelah normalisasi 2 lapis.","normalized_payload":"query=' or 1=1","ra
w_input_sample":"query=%25%32%37%25%32%30%25%34%46%25%35%32%25%32%30%25%33%31%25%33%44%25%
33%31","status":403}
```

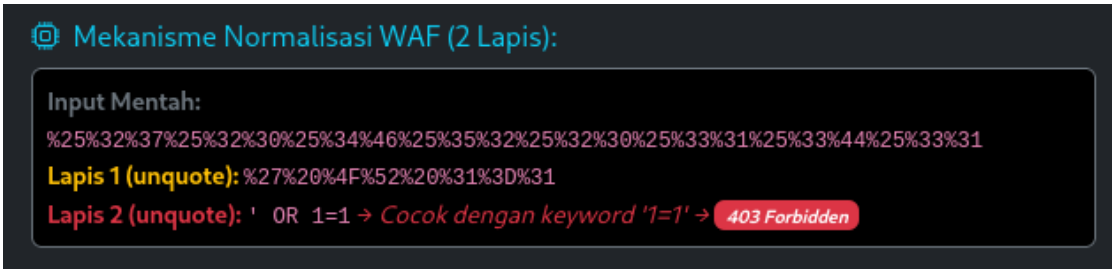
Sistem keamanan berhasil melakukan normalisasi terhadap payload yang disamakan (evasion menggunakan double URL encoding). Setelah didekode sebanyak dua lapis, sistem mengenali pola SQL Injection berupa keyword terlarang 1=1 dan secara otomatis menolak permintaan tersebut.

b. WAF pada sistem lab ini telah diprogram untuk melakukan "Normalisasi" (membuka sandi URL) sebelum memeriksa *keyword*. Lakukan pengujian serangan manual menggunakan browser dengan memasukkan *payload* yang disandikan (*Double URL Encoded*) ke *endpoint* `/search`.

```
┌──(kurniawan㉿Kurniawan)-[~/seclab_modul5]
└─$ curl -s -o /dev/null -w "HTTP Status: %{http_code}\n" "https://including-uses-special-clea
ning.trycloudflare.com/search?query+Annual+report"
HTTP Status: 200
```

Gambar tersebut menunjukkan hasil eksekusi perintah curl di terminal untuk mengirimkan pengujian permintaan teks normal (query+Annual+report) ke endpoint /search pada lab SecLab, yang menghasilkan respons HTTP Status: 200 (berhasil).

c. Lampirkan *screenshot* yang menunjukkan bahwa Anda diblokir dengan *HTTP 403 Forbidden*.



d. **Konsep Defensif:** Mengapa sebuah *Web Application Firewall (WAF)* WAJIB melakukan proses Normalisasi Data (*Data Normalization / Decoding*) terlebih dahulu sebelum mencocokkannya dengan *Rule/Signature* keamanan?

Sebuah Web Application Firewall (WAF) WAJIB melakukan proses normalisasi data (Data Normalization / Decoding) terlebih dahulu sebelum mencocokkannya dengan aturan (rule/signature) keamanan karena beberapa alasan krusial berikut:

1. Mencegah Teknik Penghindaran (Evasion Techniques): Penyerang sering kali menyamarkan payload berbahaya (seperti skrip atau perintah SQL) menggunakan teknik pengkodean berulang, seperti Double URL Encoding (misalnya %2527 menjadi %27 lalu menjadi '), Unicode, atau Hex. Tanpa normalisasi, WAF hanya akan membaca teks mentah yang tampak tidak berbahaya, padahal aplikasi di backend akan mendekodenya kembali dan mengeksekusi serangan tersebut.

2. Menyamakan Persepsi (Interpretation Mismatch): WAF dan server aplikasi web harus memproses input dengan cara yang sama. Jika WAF tidak mendekode input terlebih dahulu sementara server aplikasi melakukannya, akan terjadi celah perbedaan interpretasi di mana penyerang dapat melewati pertahanan WAF dengan mudah.

3. Akurasi Pencocokan Pola (Signature Matching): Normalisasi mengubah berbagai variasi bentuk karakter ke dalam format standar (plaintext). Hal ini memungkinkan mesin pemindai WAF mengenali pola kata kunci terlarang (seperti 1=1 atau script) secara akurat, terlepas dari bagaimana cara penyerang mencoba menyembunyikannya.

D. FORMAT LAPORAN DAN PENGUMPULAN Laporan insiden (*Incident Report*) disajikan secara profesional, ringkas, dan detail tanpa menggunakan format Bab akademis. Sistematika laporan memuat:

1. **Halaman Sampul:** Nama Kelompok, Daftar Nama Anggota, Nama Divisi, Nama Program.
2. **Ringkasan Eksekutif (Executive Summary):** Gambaran singkat (1-2 paragraf) mengenai insiden yang terjadi dan *highlight* tindakan mitigasi yang telah dilakukan tim. 3. **Analisis Temuan (Incident Findings):** Penjabaran teknis mengenai investigasi lalu lintas DoS (menjawab Bagian 1) dan dekonstruksi *payload evasion* (menjawab Bagian 2). Wajib menyertakan cuplikan baris log asli sebagai barang bukti forensik. 4. **Tindakan Mitigasi (Remediation Actions):** Penjelasan langkah konfigurasi yang dilakukan pada WAF (menjawab Bagian 3). Wajib menyertakan *screenshot* pembuktian respons server (HTTP 429 dan HTTP 403).
5. **Kesimpulan:** Penilaian singkat mengenai status keamanan postur sistem saat ini setelah mitigasi diterapkan.
6. **Format Penamaan Berkas:** Laporan diekspor ke PDF dengan nama: `[Nama Kelompok]_[Nama Divisi]_Tugas Modul 5.pdf`

Total Nilai Maksimal: 100 poin *(Nilai ditekankan pada kedalaman argumen teknis forensik log dan pemahaman filosofi WAF Normalization, bukan sekadar berhasil menyalakan limit).*